

7th August, 2026

Java: Decision Making, Loops, Bitwise Operators, and Arrays — Detailed Notes

Part 1: Decision Making

Decision-making constructs let a program choose which statements to execute based on a condition. In Java, every conditional expression must resolve to a `boolean` (`true` / `false`) — unlike C/C++, integers cannot be used directly as conditions (`if (1)` is a compile error).

1.1 The `if` Statement

Concept: The simplest branching construct. The block executes only if the condition is `true`; otherwise control skips to the statement after the block.

```
int temperature = 40;

if (temperature > 35) {
    System.out.println("It's hot outside");
}
```

How it works internally: The JVM evaluates the boolean expression, and the bytecode instruction `ifeq` / `ifne` (or similar) branches to a different instruction offset depending on the result. There is no fallthrough — execution either enters the block or jumps past it entirely.

Why braces matter even for one-line bodies:

```
if (temperature > 35)
    System.out.println("Hot");
    System.out.println("This always runs, regardless of temperature");
```

Only the first statement is part of the `if`. The second line executes unconditionally because indentation has no syntactic meaning in Java. Omitting braces is a classic source of bugs — always use them.

1.2 `if-else` and `if-else-if`

Concept: `else` provides an alternative path when the condition is `false`. Chaining `else if` allows testing multiple mutually exclusive conditions in order — the first one that matches wins, and the rest are skipped.

```
int marks = 72;

if (marks >= 90) {
    System.out.println("Grade A");
} else if (marks >= 75) {
    System.out.println("Grade B");
} else if (marks >= 60) {
    System.out.println("Grade C");
} else {
    System.out.println("Fail");
}
```

Why order matters: Conditions are evaluated top-to-bottom, and evaluation stops at the first true branch. If you had written `marks >= 60` before `marks >= 90`, every mark of 90+ would incorrectly fall into the "Grade C" bucket, because 90 also satisfies `>= 60` and that check is reached first. Always order range checks from most restrictive to least restrictive (or vice versa, consistently).

1.3 Nested `if`

Concept: An `if / else` block can contain another `if / else` inside it, allowing compound logic that depends on more than one condition being tested in sequence rather than combined with `&& / ||`.

```
int age = 20;
boolean hasLicense = true;

if (age >= 18) {
    if (hasLicense) {
        System.out.println("Can drive");
    } else {
        System.out.println("Get a license first");
    }
} else {
    System.out.println("Too young");
}
```

The dangling-else problem: When braces are omitted, an `else` always binds to the *nearest* unmatched `if`, which can be surprising:

```

if (a > 0)
    if (b > 0)
        System.out.println("both positive");
else
    System.out.println("a is not positive"); // WRONG - actually binds to
inner if!

```

Despite the indentation suggesting otherwise, this `else` pairs with `if (b > 0)`, not `if (a > 0)`. This is exactly why explicit braces are considered good practice in nested conditionals.

1.4 The Ternary (Conditional) Operator `?:`

Concept: A compact expression-form of `if-else` that *produces a value* rather than executing a statement block. Syntax: `condition ? valueIfTrue : valueIfFalse`.

```

int a = 10, b = 20;
int max = (a > b) ? a : b;

```

Why use it: It's an expression, so it can be embedded inline — inside method calls, assignments, or string concatenation — where a full `if-else` statement syntactically cannot go:

```

System.out.println("You are " + (age >= 18 ? "an adult" : "a minor"));

```

When to avoid it: Nesting ternaries (`a ? b : (c ? d : e)`) is legal but hurts readability sharply. If the logic needs more than one level, prefer a normal `if-else-if` chain or a `switch`.

1.5 The `switch` Statement

Concept: `switch` compares one variable against several constant values and jumps directly to the matching case, avoiding a long `if-else-if` chain when checking equality against discrete values.

```

int day = 3;
String name;

switch (day) {
    case 1:
        name = "Monday";
        break;
    case 2:

```

```

        name = "Tuesday";
        break;
    case 3:
        name = "Wednesday";
        break;
    default:
        name = "Invalid";
}

```

Supported types: `byte`, `short`, `char`, `int` and their wrapper classes (`Byte`, `Short`, `Character`, `Integer`), `String` (since Java 7), and `enum` types. `long`, `float`, `double`, and `boolean` are *not* allowed.

Why `break` is required: Without it, execution "falls through" to the next case regardless of whether its label matches — the JVM's `tableswitch` / `lookupswitch` bytecode simply jumps to the matching offset and then runs sequentially unless told to jump out. This is a deliberate design choice inherited from C, useful for grouping cases:

```

switch (day) {
    case 6:
    case 7:
        System.out.println("Weekend");
        break;
    default:
        System.out.println("Weekday");
}

```

Here, `case 6` has no body, so it falls through into `case 7`'s body — both values print "Weekend". Forgetting a `break` where fallthrough is *not* intended is one of the most common bugs in beginner Java code.

`default`: Optional, but recommended — it catches any value not explicitly listed. It can appear anywhere in the switch (not necessarily last), though placing it last is conventional.

1.6 Switch Expressions (Java 14+)

Concept: A modernized form of `switch` that is itself an *expression* (produces a value directly, like the ternary operator) and uses `->` instead of `case ... :`. It eliminates fallthrough by default, making it safer.

```

int day = 3;
String name = switch (day) {
    case 1, 7 -> "Weekend-ish";
    case 2, 3, 4, 5, 6 -> "Weekday";
}

```

```
    default -> "Invalid";  
};
```

Key differences from classic `switch`:

- No `break` needed — each arm executes only its own code, then exits automatically.
- Multiple labels can share one arm using commas (`case 1, 7 ->`).
- The compiler enforces *exhaustiveness* for `enum` switches (and requires a `default` otherwise), catching missing cases at compile time.

Using `yield` for multi-statement arms: If an arm needs more than one expression, use a block with `yield` to specify the returned value:

```
double discount = switch (quantity) {  
    case 1, 2, 3 -> 0.0;  
    default -> {  
        double d = quantity * 0.5;  
        yield Math.min(d, 20.0);  
    }  
};
```

1.7 Pattern Matching for `instanceof` (Java 16+)

Concept: Traditionally, checking a type and then casting required two steps:

```
if (obj instanceof String) {  
    String s = (String) obj;    // manual cast  
    System.out.println(s.length());  
}
```

Pattern matching combines the check and the cast into one expression, binding a new variable automatically only within the scope where the check is known to be true:

```
if (obj instanceof String s && s.length() > 3) {  
    System.out.println(s.toUpperCase());  
}
```

Here `s` is only in scope (and only guaranteed non-null and of type `String`) inside the `if` block and in any code path where the compiler can prove the condition held — reducing redundant casts and `ClassCastException` risk.

Part 2: Loops

Loops repeat a block of code while a condition holds, avoiding manual repetition. Java offers four looping constructs, each suited to different scenarios.

2.1 The `for` Loop

Concept: Best when the number of iterations is known or countable in advance. It packs initialization, condition-checking, and increment/decrement into one line, keeping loop-control logic in a single visible place.

```
for (int i = 0; i < 5; i++) {  
    System.out.println(i);  
}
```

Anatomy:

1. **Initialization** (`int i = 0`) — runs once, before the loop starts.
2. **Condition** (`i < 5`) — checked before every iteration, including the first. If `false`, the loop body never runs.
3. **Update** (`i++`) — runs after each iteration, before the condition is re-checked.

Execution order: init → check condition → run body → run update → check condition → run body → update → ... until condition is `false`.

Multiple variables: Java allows comma-separated init and update expressions (but only one condition):

```
for (int i = 0, j = 10; i < j; i++, j--) {  
    System.out.println(i + " " + j);  
}
```

This is useful for two-pointer patterns (e.g., converging from both ends of an array).

2.2 Enhanced `for` (for-each)

Concept: A simplified syntax for iterating over every element of an array or any `Iterable` (like `List`, `Set`) without manually managing an index variable. It reads as "for each element `n` in `nums`, do...".

```
int[] nums = {2, 4, 6, 8};  
for (int n : nums) {
```

```
    System.out.println(n);  
}
```

Trade-offs versus the classic `for` :

- **Pros:** Less error-prone (no off-by-one risk on the index), more readable when the index itself isn't needed.
- **Cons:** No access to the index (so you can't easily print "element 2 is 6"), cannot skip elements or iterate backwards, and for primitive arrays the loop variable holds a *copy* of each value — reassigning it does not modify the underlying array.

```
int[] arr = {1, 2, 3};  
for (int x : arr) {  
    x = 100;    // has no effect on arr - x is a local copy  
}
```

2.3 The `while` Loop

Concept: Repeats a block *as long as* a condition remains true, checked **before** each iteration. Best when the number of iterations isn't known ahead of time and depends on some external or evolving state (e.g., reading input until a sentinel value appears).

```
int i = 0;  
while (i < 5) {  
    System.out.println(i);  
    i++;  
}
```

Zero-execution possibility: Because the condition is checked first, the body may never execute at all if the condition starts `false` :

```
int n = 10;  
while (n < 5) {  
    System.out.println("never printed");  
}
```

2.4 The `do-while` Loop

Concept: Like `while` , but checks the condition **after** the body runs, guaranteeing at least one execution. Useful for menu-driven programs or input validation, where you must perform an action once before you can even evaluate whether to repeat it (e.g., "ask for input, then re-ask if invalid").

```
int i = 0;
do {
    System.out.println(i);
    i++;
} while (i < 5);
```

Syntax note: Unlike `while`, `do-while` requires a trailing semicolon after the condition.

2.5 Loop Control: `break` and `continue`

break : Immediately terminates the *entire* enclosing loop (or `switch`), and execution resumes at the statement right after the loop.

```
for (int i = 0; i < 10; i++) {
    if (i == 5) break;
    System.out.println(i);    // prints 0,1,2,3,4
}
```

continue : Skips only the *rest of the current iteration* and jumps straight to the update step (in a `for` loop) or the condition check (in a `while` / `do-while`) — it does not exit the loop.

```
for (int i = 0; i < 10; i++) {
    if (i % 2 == 0) continue;
    System.out.println(i);    // prints only odd numbers: 1,3,5,7,9
}
```

Conceptual distinction: `break` says "I'm done with this loop entirely." `continue` says "I'm done with *this pass*, but keep looping."

2.6 Labeled `break` / `continue`

Concept: In nested loops, a plain `break` / `continue` only affects the *innermost* loop. Labels let you name an outer loop and control it directly from inside a nested loop, avoiding boolean "flag" variables that would otherwise be needed to escape multiple levels.

```
outer:
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        if (j == 1) continue outer;    // skips to next i, abandoning inner loop
        if (i == 2) break outer;        // exits both loops entirely
        System.out.println(i + " " + j);
    }
}
```

```
}  
}
```

Walkthrough: For `i=0`: `j=0` prints "0,0", then `j=1` triggers `continue outer` — the inner loop is abandoned and `i` advances to 1. Same pattern for `i=1`. When `i=2`, `j=0` triggers `break outer` before anything is printed for `i=2`, terminating both loops immediately.

2.7 Infinite Loops

Concept: A loop with a condition that never becomes false, used deliberately for constructs like servers, event loops, or games where the exit condition is decided *inside* the loop body via `break` or `return` rather than in the loop header.

```
for (;;) {  
    // all three for-clauses are optional; an empty condition defaults to true  
}  
  
while (true) {  
    // ...  
}
```

Both forms are equivalent; `while (true)` is generally considered more readable.

Part 3: Bitwise Operators

Bitwise operators manipulate the individual binary digits (bits) of integer types (`byte`, `short`, `char`, `int`, `long`). They operate directly on the two's-complement binary representation, making them fast and useful for flags, masks, and low-level numeric tricks.

3.1 Background: Two's Complement

Java integers are stored in two's-complement form: the most significant bit (MSB) represents the sign (0 = positive, 1 = negative), and negative values are represented as the bit-inversion of the positive value plus 1. For example, for a 4-bit illustration: `5 = 0101`, `-5 = 1011` (invert `0101` → `1010`, add 1 → `1011`). Understanding this is essential for interpreting the sign-related behavior of shifts below.

3.2 AND (`&`), OR (`|`), XOR (`^`), NOT (`~`)

Operator	Rule
&	Result bit is 1 only if both input bits are 1
\	Result bit is 1 if either input bit is 1
^	Result bit is 1 if the input bits differ
~	Flips every bit (0 → 1 , 1 → 0)

```
int a = 12;    // binary: 0000 1100
int b = 10;    // binary: 0000 1010

System.out.println(a & b);    // 0000 1000 = 8
System.out.println(a | b);    // 0000 1110 = 14
System.out.println(a ^ b);    // 0000 0110 = 6
System.out.println(~a);       // inverts all 32 bits = -13
```

Why ~a gives -13: ~a is mathematically equivalent to $-a - 1$. Flipping every bit of a two's-complement number is the same as negating it and subtracting one, so $\sim 12 = -12 - 1 = -13$.

Practical uses:

- & is used to *mask* bits — isolate specific bits while zeroing out the rest (e.g., `value & 0xFF` extracts the lowest byte).
- | is used to *set* bits — turn specific bits on without disturbing others.
- ^ is used to *toggle* bits, and has a famous side property: $x \wedge x = 0$ and $x \wedge 0 = x$, which enables swapping two variables without a temporary variable:

```
int x = 5, y = 9;
x = x ^ y;    // x = 5^9
y = x ^ y;    // y = (5^9)^9 = 5
x = x ^ y;    // x = (5^9)^5 = 9
// Now x = 9, y = 5 - swapped, with no temp variable
```

3.3 Shift Operators (<< , >> , >>>)

Left shift <<: Shifts all bits left by *n* positions, filling vacated positions on the right with 0. Each left-shift by 1 is equivalent to multiplying by 2 (as long as no overflow occurs).

```
int n = 5;    // 0000 0101
System.out.println(n << 2); // 0001 0100 = 20    (5 × 2² = 20)
```

Signed right shift `>>` : Shifts bits right by n positions, filling vacated positions on the left with a copy of the **sign bit** (arithmetic shift). This preserves the sign of the number, making `>>` equivalent to integer division by a power of 2, rounded toward negative infinity.

```
int n = 5;
System.out.println(n >> 1);    // 0000 0010 = 2    (5 ÷ 2, floored)

int neg = -8;
System.out.println(neg >> 1);  // -4 - sign bit (1) is copied into the new
leftmost bits
```

Unsigned right shift `>>>` : Shifts bits right, but always fills vacated positions with 0, regardless of the sign bit (logical shift). For positive numbers this behaves identically to `>>`; for negative numbers it produces a large positive number because the sign bit is no longer treated specially.

```
int neg = -8;                // 1111...11111000 in 32-bit binary
System.out.println(neg >> 1); // -4 (sign-preserving)
System.out.println(neg >>> 1); // 2147483644 (sign bit becomes an ordinary
0 bit)
```

Why this distinction matters: `>>>` exists specifically because Java has no *unsigned* integer types (unlike C's `unsigned int`). When you need to treat a 32-bit pattern as a raw unsigned bit sequence (e.g., in hashing or checksum code), `>>>` avoids the sign bit "bleeding into" your shifted value.

Shift amount wraps around: For `int` operands, the shift count is taken modulo 32 (so `n << 33` behaves like `n << 1`); for `long`, modulo 64.

3.4 Compound Bitwise Assignment

Bitwise operators can be combined with `=` just like arithmetic operators, applying the operation and reassigning in one step:

```
int flags = 0;
flags |= 1;    // equivalent to: flags = flags | 1;    (set a bit)
flags &= ~2;   // equivalent to: flags = flags & ~2;   (clear a bit)
flags ^= 4;    // equivalent to: flags = flags ^ 4;    (toggle a bit)
```

This pattern is the backbone of "bit flag" systems, where a single integer packs many independent boolean settings, one per bit, instead of using several separate `boolean` variables.

3.5 Common Bit Manipulation Idioms (with reasoning)

```
// Check even/odd: the lowest bit of any odd number is 1, of any even number
// is 0.
boolean isEven = (n & 1) == 0;

// Check power of two: a power of two has exactly one set bit (e.g., 8 =
// 1000).
// Subtracting 1 flips that bit and all bits below it (7 = 0111).
// ANDing the two together yields 0 only when n had a single set bit.
boolean isPowerOfTwo = n > 0 && (n & (n - 1)) == 0;

// Get the i-th bit: shift the target bit down to position 0, then mask with
// 1.
int get = (n >> i) & 1;

// Set the i-th bit: OR with a mask that has only bit i turned on.
int set = n | (1 << i);

// Clear the i-th bit: AND with a mask that has every bit set EXCEPT bit i.
int clear = n & ~(1 << i);

// Toggle the i-th bit: XOR with a mask that has only bit i turned on.
int toggle = n ^ (1 << i);

// Count set bits (Brian Kernighan's algorithm):
// n & (n-1) always clears the LOWEST set bit of n, so counting how many
// times this can be done before n becomes 0 gives the total set-bit count.
int count = 0;
int temp = n;
while (temp != 0) {
    temp &= (temp - 1);
    count++;
}
// Java also provides a built-in, hardware-accelerated equivalent:
int builtin = Integer.bitCount(n);
```

3.6 Logical (&& , ||) vs Bitwise-on-Boolean (& , |)

Java allows `&` and `|` to operate on `boolean` operands too, but the behavior differs subtly from `&&` / `||`:

```
boolean p = true, q = false;
boolean r1 = p && q; // short-circuit: since p && q depends on q too, q IS
```

evaluated here

```
boolean r2 = p & q;    // non-short-circuit: always evaluates both p and q
```

Short-circuiting explained: `&&` and `||` skip evaluating the right-hand operand when the result is already determined by the left-hand operand alone (`false && x` is always `false` ; `true || x` is always `true`). This matters when the right-hand expression has side effects or could throw an exception:

```
// Safe: short-circuiting prevents calling .length() on a null reference
if (str != null && str.length() > 0) { ... }
```

```
// Unsafe: & always evaluates both sides, so if str is null this throws
NullPointerException
if (str != null & str.length() > 0) { ... }
```

`& / ||` on booleans are rarely used in practice, mainly reserved for cases where evaluating both sides deliberately (e.g., to force a method call's side effect to always run) is desired.

Part 4: Arrays

An array is a fixed-size, ordered collection of elements of the same type, stored in contiguous memory and accessed by a zero-based integer index. Arrays are Java's most basic data structure and are themselves objects (they live on the heap and have a `.length` field).

4.1 Declaration and Initialization

```
int[] arr1;                // declaration only - arr1 currently holds
null                       //
arr1 = new int[5];          // allocation - creates an array of 5 ints,
all default 0              //

int[] arr2 = new int[]{1, 2, 3, 4, 5}; // explicit allocation + initializer
int[] arr3 = {1, 2, 3, 4, 5};          // shorthand initializer (declaration-
site only)

int arr4[] = {1, 2, 3};              // C-style syntax - legal but not idiomatic in
Java
```

Default values on allocation: When an array is created with `new` , every slot is automatically filled with a type-appropriate default — `0` for numeric primitives, `false` for `boolean` , the `null`

character `'\u0000'` for `char`, and `null` for any reference/object type. This differs from local variables, which are *not* auto-initialized and must be explicitly assigned before use.

length is a field, not a method: `arr.length` (no parentheses) — a common source of confusion for those coming from `String.length()` or `List.size()`, both of which *are* method calls.

4.2 Accessing and Modifying Elements

```
int[] arr = {10, 20, 30};
System.out.println(arr[0]); // 10 - first element, index 0
arr[1] = 99;                // array is now {10, 99, 30}
```

Bounds checking: Java performs bounds checking on *every* array access at runtime. Accessing `arr[3]` on a 3-element array throws `ArrayIndexOutOfBoundsException` rather than silently reading adjacent memory (as could happen in C). There is no negative indexing (`arr[-1]` also throws this exception) — this differs from languages like Python.

4.3 Iterating Arrays

Index-based (classic `for`): Needed when you require the index itself, need to iterate backwards, or need to modify elements in place.

```
int[] arr = {1, 2, 3, 4};
for (int i = 0; i < arr.length; i++) {
    System.out.println("Index " + i + ": " + arr[i]);
}
```

Value-based (for-each): Cleaner when only the values matter.

```
for (int val : arr) {
    System.out.println(val);
}
```

4.4 Multidimensional Arrays

Concept: Java doesn't have "true" multidimensional arrays in the sense of a single contiguous block with a fixed row-width (as in C); instead, a 2D array is really an *array of array references* — each row is its own independently allocated 1D array.

```
int[][] matrix = new int[3][4]; // 3 rows, each with 4 columns
matrix[0][0] = 1;
```

```

int[][] grid = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};

for (int i = 0; i < grid.length; i++) {           // grid.length = number of
rows
    for (int j = 0; j < grid[i].length; j++) {     // grid[i].length = columns
in row i
        System.out.print(grid[i][j] + " ");
    }
    System.out.println();
}

```

Jagged arrays: Because each row is a separate array object, rows can have different lengths — this is called a "jagged" array and is fully legal (and sometimes intentional, e.g., representing a triangular structure):

```

int[][] jagged = new int[3][];           // allocate 3 row-references, no columns
yet
jagged[0] = new int[]{1};                // row 0 has 1 element
jagged[1] = new int[]{1, 2};             // row 1 has 2 elements
jagged[2] = new int[]{1, 2, 3};          // row 2 has 3 elements

```

4.5 Arrays Are Objects (Reference Semantics)

Concept: Unlike primitive variables, an array variable holds a *reference* (essentially a pointer) to the array's data on the heap, not the data itself. Assigning one array variable to another copies the *reference*, not the contents — both variables then point to the exact same underlying array.

```

int[] a = {1, 2, 3};
int[] b = a;           // b now points to the SAME array object as a
b[0] = 99;
System.out.println(a[0]); // prints 99 – because a and b are aliases of one
array

```

Making an actual copy: To get an independent array with the same values, use `.clone()` or `Arrays.copyOf()`:

```
int[] c = a.clone();    // c is a NEW array with the same element values
c[0] = 1;
System.out.println(a[0]); // still 99 – a is unaffected
```

Note that `.clone()` performs a *shallow* copy: for arrays of primitives this is a full, independent copy, but for arrays of objects, the clone contains references to the *same* objects as the original — only the outer array structure is duplicated.

4.6 The `java.util.Arrays` Utility Class

Concept: Rather than writing manual loops for common array tasks, the standard library provides `java.util.Arrays`, offering static helper methods:

```
import java.util.Arrays;

int[] arr = {5, 3, 8, 1};

Arrays.sort(arr);                // sorts in place: {1, 3, 5, 8}
int idx = Arrays.binarySearch(arr, 5); // fast search – REQUIRES the array
// be sorted first
int[] copy = Arrays.copyOf(arr, 6); // new array of length 6; extra
// slots default-filled
int[] range = Arrays.copyOfRange(arr, 1, 3); // elements from index 1
// (inclusive) to 3 (exclusive)
boolean eq = Arrays.equals(arr, copy); // compares contents, element by
// element
Arrays.fill(arr, 0);              // overwrites every element with 0
String s = Arrays.toString(arr);  // human-readable string: "[0, 0,
// 0, 0]"
String s2 = Arrays.deepToString(grid); // needed for multi-dimensional
// arrays
```

Why `Arrays.equals()` instead of `==`: `arr1 == arr2` compares whether both variables reference the *same* array object in memory, not whether their contents match. Two distinct arrays holding identical values will be `==`-unequal but `Arrays.equals()`-equal. Similarly, `System.out.println(arr)` alone does *not* print the contents — it prints the array's memory-derived hash representation (e.g., `[I@1b6d3586]`) because arrays don't override `toString()`; use `Arrays.toString()` instead.

4.7 Passing Arrays to Methods

Concept: Because arrays are objects, passing one as a method argument passes a copy of the *reference* (not the data). This means the method receives its own variable pointing to the

same array, so modifications to the array's *contents* inside the method are visible to the caller after the method returns.

```
static void doubleValues(int[] arr) {
    for (int i = 0; i < arr.length; i++) {
        arr[i] *= 2;
    }
}

int[] nums = {1, 2, 3};
doubleValues(nums);
System.out.println(Arrays.toString(nums)); // [2, 4, 6] - caller's array WAS
modified
```

But reassignment inside the method does not propagate back:

```
static void reassign(int[] arr) {
    arr = new int[]{100, 200}; // arr now points to a NEW array - only inside
    this method
}

int[] nums = {1, 2, 3};
reassign(nums);
System.out.println(Arrays.toString(nums)); // [1, 2, 3] - unchanged, caller's
reference untouched
```

The distinction: mutating `arr[i] = ...` changes shared data; `arr = new int[...]` merely repoints the local variable `arr`, leaving the caller's original reference (`nums`) pointing at the original array.

4.8 Common Pitfalls Summary

- `arr.length` is a **field**, never call it as `arr.length()`.
- No negative indexing or slicing syntax exists — `arr[-1]` and `arr[1:3]` both fail to compile/throw.
- `==` on arrays compares **references**, not contents; use `Arrays.equals()`.
- Array size is fixed at creation — there is no "append" or "resize"; use `ArrayList<T>` (from `java.util`) when the collection size needs to grow or shrink dynamically.

- `ArrayIndexOutOfBoundsException` is a *runtime* exception — the compiler cannot catch out-of-range constant indices in most cases, so bounds errors surface only when the code actually executes.